

SENAC RJ — Análise e Desenvolvimento de Sistemas

Gerenciamento Ágil de Projetos

SPRINT 5 — ANÁLISE CRÍTICA

Tema: DDD — Entendendo o Domínio do Problema

Entrega: 12/06/2026

Este documento apresenta uma análise crítica do Domain-Driven Design (DDD), discutindo suas vantagens, limitações e desafios de adoção, seguida de uma comparação com outra abordagem de modelagem de software: a **Modelagem Orientada a Dados (Data-Driven Design)**. O objetivo é avaliar o DDD de forma equilibrada, reconhecendo seu valor estratégico sem ignorar os custos reais de sua aplicação.

1. Introdução

Nas sprints anteriores, o grupo apresentou os fundamentos conceituais do DDD (Domínio, Linguagem Ubíqua, Bounded Context) e sua aplicação prática por meio da modelagem tática de um sistema fictício de Gestão de Entregas Logísticas (Entidades, Value Objects, Agregados e Repositórios). Esta sprint avança para uma etapa de reflexão crítica: avaliar em que contextos o DDD efetivamente agrega valor e em quais ele pode representar um custo desproporcional ao benefício.

O DDD é frequentemente apresentado na literatura técnica de forma excessivamente favorável, como uma solução universal para a complexidade de software. Esta análise busca contrapor esse discurso com uma visão mais realista, baseada na própria literatura fundadora do tema e em relatos práticos de adoção.

2. Vantagens do DDD

✓	Alinhamento negócio-tecnologia A Linguagem Ubíqua elimina a tradução constante entre o vocabulário de negócio e o vocabulário técnico, reduzindo mal-entendidos que normalmente só são descobertos tarde no ciclo de desenvolvimento.
✓	Modelos que refletem a realidade do negócio Como o modelo de software nasce da colaboração direta com especialistas de domínio, ele tende a capturar regras de negócio com mais fidelidade do que modelos derivados puramente da estrutura de dados.
✓	Fronteiras claras em sistemas complexos O conceito de Bounded Context permite dividir sistemas grandes em partes gerenciáveis, cada uma com seu próprio modelo consistente, evitando o efeito de 'modelo único gigante' que se torna contraditório à medida que o sistema cresce.
✓	Facilita a adoção de microsserviços Bounded Contexts bem definidos se traduzem naturalmente em fronteiras de microsserviços, já que cada contexto tem seu próprio modelo de dados, linguagem e ciclo de vida de deploy.
✓	Evolução controlada do modelo Agregados e invariantes explícitas tornam visíveis as regras de negócio que, em outras abordagens, ficariam implícitas e dispersas pela camada de aplicação.

3. Limitações e Desafios

!	Alto custo de entrada O DDD exige tempo significativo de imersão no domínio antes de qualquer linha de código ser escrita com confiança. Em contextos de prazo curto ou MVPs, esse investimento inicial pode não se justificar.
!	Dependência de especialistas de domínio disponíveis A construção da Linguagem Ubíqua depende de acesso contínuo e qualificado a domain experts. Em organizações onde esses especialistas têm pouca disponibilidade, o modelo tende a ser construído com suposições incorretas.
!	Curva de aprendizado da equipe técnica Conceitos como Bounded Context, Agregado e Anti-Corruption Layer não são triviais. Equipes sem experiência prévia frequentemente aplicam os padrões de forma incorreta, criando agregados excessivamente grandes ou bounded contexts mal delimitados, o que anula os benefícios da abordagem.
!	Overengineering em domínios simples Para sistemas de domínio simples ou predominantemente CRUD, aplicar todo o aparato do DDD tático (entidades, value objects, agregados, repositórios, serviços de domínio) pode adicionar complexidade acidental sem retorno proporcional.
!	Dificuldade de medir o retorno Diferentemente de métricas ágeis como velocity ou lead time, os benefícios do DDD — modelos mais expressivos, menor acoplamento entre contextos — são qualitativos e de longo prazo, dificultando a justificativa do investimento perante stakeholders focados em entregas de curto prazo.
!	Risco de fragmentação excessiva Bounded Contexts mal dimensionados podem levar a uma proliferação de microsserviços pequenos e fortemente interdependentes, problema conhecido como 'distributed monolith', que combina a complexidade operacional de sistemas distribuídos com o acoplamento de um monolito.

4. Quando o DDD Compensa o Investimento

A literatura especializada — e a própria obra de Eric Evans — é enfática em um ponto frequentemente ignorado: o DDD não deve ser aplicado uniformemente a todo o sistema. Ele é mais valioso quando direcionado ao **Core Domain**, deixando Generic Domains serem resolvidos com soluções prontas ou de terceiros.

Cenário	DDD compensa?	Justificativa
---------	---------------	---------------

Domínio de negócio complexo e com diferencial competitivo	Sim	O investimento no modelo se traduz em vantagem estratégica real.
Sistema CRUD simples ou administrativo interno	Não	O overhead conceitual supera o ganho; abordagens mais simples bastam.
Equipe pequena, sem acesso a domain experts	Parcial	Pode-se aplicar a Linguagem Ubíqua sem o aparato tático completo.
Migração de monolito para microsserviços	Sim	Bounded Contexts orientam fronteiras de serviço de forma mais segura que divisões puramente técnicas.

5. Comparação com Outra Abordagem de Modelagem

Para contextualizar as escolhas do DDD, comparamos a abordagem com a **Modelagem Orientada a Dados (Data-Driven Design)**, ainda amplamente utilizada em sistemas corporativos tradicionais, na qual o modelo de software nasce da estrutura do banco de dados em vez do comportamento do domínio de negócio.

Critério	DDD (Domain-Driven Design)	Modelagem Orientada a Dados
Ponto de partida	Comportamento e regras do domínio de negócio.	Estrutura de tabelas e relacionamentos do banco.
Foco do modelo	Entidades ricas com lógica de negócio encapsulada.	Entidades 'anêmicas' — apenas dados, sem comportamento.
Onde vive a regra de negócio	Dentro do Agregado, próxima aos dados que protege.	Em serviços/camadas de aplicação separadas dos dados.
Velocidade inicial de desenvolvimento	Mais lenta — exige descoberta de domínio.	Mais rápida — modelo deriva direto do esquema de dados.
Manutenibilidade em domínios complexos	Alta — mudanças de regra ficam isoladas no agregado.	Baixa — regras se dispersam por múltiplas camadas/serviços.
Adequação a sistemas simples/CRUD	Baixa — overhead conceitual desnecessário.	Alta — abordagem direta e eficiente.
Acoplamento ao banco de dados	Baixo — o domínio é independente da persistência.	Alto — mudanças no schema impactam todo o sistema.

No sistema de Gestão de Entregas Logísticas modelado pelo grupo, essa diferença fica evidente: na abordagem DDD, a regra *'um Pedido deve ter ao menos um Item'* vive dentro da entidade Pedido e é impossível violá-la sem passar pela raiz do agregado. Na modelagem orientada a dados, essa mesma regra precisaria ser validada manualmente em cada serviço que manipula a tabela de pedidos, criando risco de inconsistência caso um novo serviço seja adicionado e esqueça a validação.

6. Considerações Finais

O DDD não é uma metodologia superior em termos absolutos, mas uma ferramenta cujo valor é fortemente dependente do contexto de aplicação. Seus benefícios — alinhamento entre negócio e tecnologia, modelos expressivos e fronteiras claras — se manifestam principalmente em domínios complexos e estratégicos, enquanto seu custo de adoção pode ser proibitivo em sistemas simples ou em equipes sem maturidade técnica suficiente.

A principal lição extraída desta análise crítica é que a decisão de adotar DDD deveria ser tomada por subdomínio, e não para o sistema como um todo — aplicando todo o rigor tático ao Core Domain e reservando abordagens mais simples, como a modelagem orientada a dados, para os Generic e Supporting Domains.